Protein Circuit Topology Plugin - Complete API Documentation

This document reflects the **current plugin/runtime source tree** and lists callable entry points by module category.

Total Callable Entry Points: 119 **Python Files With Callables:** 37

Table of Contents

Calculating Functions

`energy_cmap(index, numbering, res_names, protid, potential_sign='-')`

Module: `functions/calculating/energy_cmap.py`

Applies an energy filter to an existing residue contact map based on a potential matrix.

`get_cmap(chain, level='chain', cutoff_distance=4.5, cutoff_numcontacts=5, exclude_neighbour=3)`

Module: `functions/calculating/get_cmap.py`

Creates a residue-residue contact map (as a list of contacts) for a single chain or a whole model.

`get_matrix(index, protid)`

Module: `functions/calculating/get_matrix.py`

Creates a topological relationship matrix for a residue contact map.

`get_stats(mat)`

Module: `functions/calculating/get_stats.py`

Calculates the percentage of entangled contacts (Parallel and Cross) further along the diagonal.

`length_filter(index, distance, mode='<')`

Module: `functions/calculating/length_filter.py`

Filters contact indices based on sequence separation distance.

`local_ct(index, mat, numbering)`

Module: `functions/calculating/local_ct.py`

Calculates local circuit topology statistics for each residue.

Plotting Functions

`circuit_plot(index, protid, numbering)`

Module: `functions/plots/circuit_plot.py`

Plots the circuit topology of a protein as a series of arcs.

`local_topology_plot(index, mat, numbering, protid, siteid, relation)`

Module: `functions/plots/local_topology_plot.py`

`matrix_plot(mat, protid)`

Module: `functions/plots/matrix_plot.py`

Plots the topological relationship matrix for a single chain.

`matrix_plot_model(mat, protid)`

Module: `functions/plots/matrix_plot_model.py`

Plots the topological relationship matrix for a whole model (multiple chains).

`autopct_funct(pct)`

Module: `functions/plots/stats_plot.py`

`stats_plot(entangled, psc, protid)`

Module: `functions/plots/stats_plot.py`

Plots the fraction of entangled contacts versus distance from

Importing Functions

`retrieve_chain(input_file, chainid=0)`

Module: `functions/importing/retrieve_chain.py`

Retrieves a specific chain from a PDB or MMCIF file.

Exporting Functions

`export_cmap3(index, protid, numbering, output_dir)`

Module: `functions/exporting/export_cmap3.py`

Exports a residue contact map (as a binary matrix) to a CSV file.

`export_mat(index, mat, protid, output_dir)`

Module: `functions/exporting/export_mat.py`

Exports the topological relationship matrix to a CSV file.

`export_psc(psclist, output_dir)`

Module: `functions/exporting/export_psc.py`

Exports the counts of Parallel, Series, and Cross contacts (and others) to a CSV file.

Analysis Functions

`run_local_ct(self)`

Module: `analysis/local_ct_analysis.py`

Runs the local circuit topology analysis based on user-selected parameters.

`run_multi_analysis(self)`

Module: `analysis/multiple_file_analysis.py`

Runs the multi-file circuit topology analysis.

`run_standard_analysis(self)`

Module: `analysis/single_file_analysis.py`

Runs the standard single-file circuit topology analysis.

`run_single_frame_analysis(self)`

Module: `analysis/single_frame_analysis.py`

Runs circuit topology analysis for a single frame of a trajectory or a single PDB file from a directory.

`toggle_frame_controls(self, enabled)`

Module: `analysis/single_frame_analysis.py`

Toggles the enabled state of the frame selector and run button.

`visualize_molecule(self, contact_type)`

Module: `analysis/visualization.py`

Visualizes the circuit topology on the selected molecule in PyMOL by coloring residues based on contact density.

Utility Functions

`clear_selected_local_file(self)`

Module: `utils/clear_file.py`

Clears the currently selected local file and updates the UI.

`clear_selected_single_file(self)`

Module: `utils/clear_file.py`

Clears the currently selected single file and updates the UI.

`_choose_output_dir(self, attr_name, label)`

Module: `utils/directory.py`

Shared helper: open a directory dialog and store the result.

`_load_structure_file(self, label, attr_file, attr_obj)`

Module: `utils/directory.py`

Shared helper: open a file dialog, load into PyMOL, check for non-polymer atoms.

`choose_file(self)`

Module: `utils/directory.py`

`choose_input_dir_multi(self)`

Module: `utils/directory.py`

Opens directory dialog to select the input directory containing PDBs for multi-file analysis.

`choose_local_file(self)`

Module: `utils/directory.py`

`choose_local_output_dir(self)`

Module: `utils/directory.py`

`choose_output_dir(self)`

Module: `utils/directory.py`

`choose_output_dir_multi(self)`

Module: `utils/directory.py`

`set_label_text_elided(file_path, label)`

Module: `utils/directory.py`

Sets the text of a QLabel to an elided version of the file path if it's too long.

`get_folding_score(mat, index, numbering)`

Module: `utils/folding_score.py`

Calculate the folding score based on the given relations, using topology data.

`get_local_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for local analysis from the GUI.

`get_multiple_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for multi-file analysis from the GUI.

`get_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for single-file analysis from the GUI.

`get_vis_vals(self)`

Module: `utils/get_values.py`

Retrieves visualization parameters from the GUI.

`_poll_pymol_objects(self)`

Module: `utils/helpers.py`

Single poll that refreshes both object dropdowns from one cmd call.

`init_timers(self)`

Module: `utils/helpers.py`

Initializes timers for updating object lists.

`make_info_button(tooltip)`

Module: `utils/helpers.py`

Creates a small info button with a tooltip.

`make_param_row(label_text, tooltip, spinbox)`

Module: `utils/helpers.py`

Create a standard parameter row layout with label, info button, and spinbox.

`object_exists(name)`

Module: `utils/helpers.py`

Checks if a PyMOL object exists.

`temp_pdb_export(selection, state=None)`

Module: `utils/helpers.py`

Save a PyMOL selection to a temporary PDB file, yield the path, then clean up.

`update_chain_combo_box(self)`

Module: `utils/helpers.py`

Updates the chain combo box with the chains available in the currently selected object.

`has_non_polymer_atoms()`

Module: `utils/non_polymer.py`

Checks if there are any non-polymer atoms in the PyMOL session.

`new_file_has_non_polymer_atoms(obj_name)`

Module: `utils/non_polymer.py`

Checks if a specific object contains non-polymer atoms.

`remove_non_polymer_atoms()`

Module: `utils/non_polymer.py`

Removes all non-polymer atoms from the PyMOL session.

`show_warning_dialog(self)`

Module: `utils/non_polymer.py`

Shows a warning dialog before removing non-polymer atoms.

`handle_local_object_change(self, obj_name)`

Module: `utils/object_change.py`

Handles changes to the selected object in the local analysis tab.

`handle_standard_object_change(self, obj_name)`

Module: `utils/object_change.py`

Handles changes to the selected object in the standard analysis tab.

`get_residue_range(self, obj_name)`

Module: `utils/residues.py`

Retrieves the residue range for each chain in the specified object.

`update_residue_range(self)`

Module: `utils/residues.py`

Updates the residue range spinbox based on the currently selected chain.

`color_by_topology(molecule_name, topology_vector, numbering, topology_type)`

Module: `utils/topology.py`

Colors a PyMOL object based on a topology vector.

`get_topology_vector(mat, index, topology_type, numbering)`

Module: `utils/topology.py`

Calculates a topology vector representing the density of a specific contact type

`export_frames_from_traj(self)`

Module: `utils/trajectory.py`

Exports each frame of the loaded trajectory as a separate PDB file.

`select_mol_file(self)`

Module: `utils/trajectory.py`

Opens a file dialog to select a structure file (PDB or CIF) for trajectory analysis.

`select_xtc_file(self)`

Module: `utils/trajectory.py`

Opens a file dialog to select a trajectory file (XTC, DCD, TRR, NC).

`update_list(self, new_objects=None)`

Module: `utils/updates.py`

Updates the list of available objects in the single-file analysis dropdown.

`update_local_list(self, new_objects=None)`

Module: `utils/updates.py`

Updates the list of available objects in the local analysis dropdown.

`update_output_widgets(self)`

Module: `utils/updates.py`

Update visibility of output widgets for single-file export options.

`update_output_widgets_local(self)`

Module: `utils/updates.py`

Updates the visibility of output widgets in the local analysis tab based on checkbox states.

`update_output_widgets_multi(self)`

Module: `utils/updates.py`

Updates the visibility of output widgets in the multi-file analysis

GUI Functions

`__init_plugin__(app=None)`

Module: `__init__.py`

Initialize the plugin within PyMOL.

`run_plugin_gui()`

Module: `__init__.py`

Create or raise the plugin GUI dialog.

`__init__(self, parent=None)`

Module: `gui_class.py`

Type: `CTDialog` method

Construct the dialog and perform UI initialization.

`_poll_pymol_objects(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Single timer callback that refreshes both object dropdowns.

`choose_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a file chooser to select a single input file (single-file tab).

`choose_input_dir_multi(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a directory chooser for selecting multiple input files (multi-file).

`choose_local_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a file chooser to select a local PDB/file for local analysis.

`choose_local_output_dir(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a directory chooser for specifying the output folder (local analysis).

`choose_output_dir(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a directory chooser for specifying the output folder (single-file).

`choose_output_dir_multi(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Open a directory chooser for specifying the output folder (multi-file).

`clear_selected_local_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Clear the selected local file control value(s).

`clear_selected_single_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Clear the selected single-file control value(s).

`export_frames_from_traj(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Export frames from an opened trajectory according to UI parameters.

`get_local_values(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Return settings from the local analysis tab UI controls.

`get_multiple_values(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Return settings from the multi-file analysis tab UI controls.

`get_residue_range(self, obj_name=None)`

Module: `gui_class.py`

Type: `CTDialog` method

Request the residue range for an object and update the UI.

`get_values(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Return settings from the single-file tab UI controls.

`get_vis_vals(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Return visualization-specific values from the UI.

`handle_local_object_change(self, obj_name=None)`

Module: `gui_class.py`

Type: `CTDialog` method

Handle changes in the local object dropdown.

`handle_standard_object_change(self, obj_name=None)`

Module: `gui_class.py`

Type: `CTDialog` method

Handle changes in the standard object dropdown.

`init_local_tab(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Initialize widgets and layout for the local analysis tab.

`init_multi_file_tab(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Initialize widgets and layout for the multi-file analysis tab.

`init_single_file_tab(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Initialize widgets and layout for the single-file analysis tab.

`init_timers(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Initialize any repeating timers used by the GUI (e.g., polling PyMOL state).

`init_ui(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Create the top-level tab widget and initialize each feature tab.

`run_local_ct(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run local circuit-topology analysis using current local-tab settings.

`run_multi_analysis(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run the batch multi-file analysis flow.

`run_single_frame_analysis(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run an analysis for the currently selected single frame.

`run_standard_analysis(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run the standard single-file analysis flow using current UI settings.

`select_mol_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Select a molecular file used by the trajectory tools.

`select_xtc_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Select an XTC (trajectory) file used by the trajectory tools.

`show_warning_dialog(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Display a warning dialog when a non-polymeric selection is detected.

`toggle_frame_controls(self, enabled)`

Module: `gui_class.py`

Type: `CTDialog` method

Enable or disable UI controls that affect frame selection.

`update_chain_combo_box(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Refresh the chain selection combobox based on the currently selected object.

`update_list(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Refresh the object list shown in the single-file tab.

`update_local_list(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Refresh the object list shown in the local-analysis tab.

`update_output_widgets(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Update widgets that display or depend on the output path (single-file).

`update_output_widgets_local(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Update widgets that display or depend on the output path (local-analysis).

`update_output_widgets_multi(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Update widgets that display or depend on the output path (multi-file).

`update_residue_range(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Apply the current residue-range widget values to the model or internal state.

`visualize_molecule(self, contact_type)`

Module: `gui_class.py`

Type: `CTDialog` method

Trigger molecule visualization in PyMOL.

`init_local_tab(self)`

Module: `tabs/local_tab.py`

Initializes the 'Local Circuit Topology' tab in the GUI.

`init_multi_file_tab(self)`

Module: `tabs/multiple_file_tab.py`

Initializes the 'Multi-File Analysis' tab in the GUI.

`init_single_file_tab(self)`

Module: `tabs/single_file_tab.py`

Initializes the 'Single-File Analysis' tab in the GUI.

Initialization Functions

`check_installed_packages(requirements_list)`

Module: `initialization_checks.py`

Checks if the required packages are installed in the current environment.

`conda_init(user_install)`

Module: `initialization_checks.py`

Initializes conda for PowerShell.

`get_requirements(req_path)`

Module: `initialization_checks.py`

Parses the requirements.yml file to get a list of required packages.

`install_failed(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Prints instructions for manual installation of dependencies if automated installation fails.

`is_conda_installed()`

Module: `initialization_checks.py`

Checks if conda is installed on the system.

`is_running_as_admin()`

Module: `initialization_checks.py`

Returns True if the current process has administrator / root privileges. On Windows calls `IsUserAnAdmin()`; on POSIX checks effective uid == 0.

`is_path_user(path)`

Module: `initialization_checks.py`

Checks if a given path is within the user's home directory.

`linux_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs installation on Linux using conda.

`mac_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs installation on macOS using conda.

`pymol_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Attempts to install dependencies using the PyMOL terminal and conda.

`register_pymol_functions()`

Module: `initialization_checks.py`

Register the plugin's core functions as PyMOL commands.

`win_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs installation on Windows using PowerShell and conda.

Function Statistics

- **Calculating Functions**: 6
- **Plotting Functions**: 6
- **Importing Functions**: 1
- **Exporting Functions**: 3
- **Analysis Functions**: 6
- **Utility Functions**: 41
- **GUI Functions**: 44
- **Initialization Functions**: 12
- **Total**: 119

Last Updated: May 21, 2026